L-Representation: Provably-Correct Finite-Precision Single-Integer Substrate for Geometric and Algebraic Computing

Formal proofs, finite-precision theorems, guard-bit optimization, segmentation, and FPGA/ASIC simulation methodology

Hung Dinh Phu Dang

Repository: `https://github.com/nahhididwin/L-Representation`

12 March 2026

Abstract

This paper is a complete rewrite and formal completion of the L-Representation (L-Rep) concept. We remove the idealized oracle assumptions used in prior sketches and replace them by fully constructive, finite-precision theorems and algorithms that can be implemented, simulated, and verified on present-day commodity hardware and in FPGA/ASIC toolchains. Concretely, the contributions are:

1. A fully constructive finite-precision isomorphism: mapping multivectors to mixed-radix packed integers and back with explicit quantization maps and proofs that arithmetic implemented with finite local accumulators and modular field packing reproduces the algebraic geometric product within provable error bounds (Theorem **??**).
2. Tight guard-bit and accumulator sizing theorems: closed-form bounds and algorithms to compute the minimum accumulator widths and storage widths per field to guarantee *no inter-field bleed* under worst-case adversarial inputs (Theorem **??**). We also derive constructive formulas for the number of guard bits required for nested operator trees and provide exact tradeoff curves (bits vs. error vs. latency) under common operator families.
3. Segmenting and mixed-precision methods that reduce the guard-bit blowup: two orthogonal constructions are proved—(i) segmentation + requantize with additive, budgeted error (Theorem **??**), and (ii) mixed-precision accumulation where local accumulators use higher precision but economize storage by selective requantization combined with stochastic/probabilistic overflow bounds (Theorem **??**).
4. A careful treatment of input quantization mappings, including necessary and sufficient conditions for "lossless" mapping, and constructive remapping strategies (gamma, affine, and dynamic range compression) when inputs are not representable exactly.
5. Practical numeric format comparisons (two's complement fixed-point, IEEE floats, Gustafson's *posit/quire*) and a quantified analysis of how switching numeric formats affects guard bits and accumulator sizing.
6. A reproducible FPGA/ASIC simulation and validation plan: Verilog reference microarchitecture, Verilator-based functional simulation, high-level fixed-point Python reference model, and HLS/Vivado/Quartus synthesis notes. We provide a self-contained RTL testbench design pattern and an algorithmic mapping from per-field parameters to estimated DSP/LUT usage and timing budgets (sample mapping and script in Appendix **??**).
7. A complete LaTeX-ready appendix with proofs (mechanized-translation friendly), runnable Python prototypes for bound computation, and recommended test vectors for exhaustive corner-case verification.

The result is a practical blueprint: no oracle/"unbounded accumulator" assumptions remain. The paper includes proofs, proof sketches where the combinatorial detail is lengthy, and a pragmatic engineering plan so the community can reproduce, verify, and extend the design.

1 Introduction

We rewrite the L-Representation idea so that every previously "oracle" assumption is replaced by finite, testable, and provable constructs. The motivating observation is simple: multivectors (as used in Geometric Algebra) are finite-dimensional vectors. Packing them into a single machine word is possible; the only obstacles are carries and quantization. We address both.

Goals and target audience. This document targets both the theoretical community (formal proofs of correctness and error bounds) and hardware engineers (concrete accumulator sizing, synthesis-friendly microarchitecture patterns, and simulation/test flows). We assume familiarity with finite-precision arithmetic, basic GA notation, and FPGA/ASIC design flows.

Structure. Sections **??**–**??** establish the finite-precision model and main theorems. Sections **??**–**??** give algorithms, proofs, and hardware mapping. Appendices provide runnable artifacts.

2 Notation and finite-precision execution model

We adopt the following finite-precision execution model. Everything is constructive and parameterized by word widths.

Definition 2.1 (L-layout)**.** An L-layout $\mathcal{L} = ((b_0, s_0, \sigma_0), \ldots, (b_{m-1}, s_{m-1}, \sigma_{m-1}))$ is a list of m field specifications where $b_i \in \mathbb{Z}_{>0}$ is storage bits (for field i), $s_i \in \{\text{unsigned}, \text{signed}\}$ is signedness, and $\sigma_i > 0$ is an integer or rational scale such that the stored integer q_i corresponds to real value $\hat{a}_i = q_i/\sigma_i$.

Note: storage bits b_i define the packing base $B_i = 2^{b_i}$. We use $o_i = \sum_{j<i} b_j$ for offsets.

Definition 2.2 (Local accumulators)**.** Each field i may be computed using a local accumulator of width $w_i \geq b_i$. Local accumulator arithmetic is assumed exact within w_i bits (i.e., using a larger integer type with width w_i). All accumulator widths are finite and must be chosen by analysis.

Definition 2.3 (Quantization map)**.** Quantize(x, σ, b, s) returns the integer q with $-2^{b-1} \leq q < 2^{b-1}$ (for signed) or $0 \leq q < 2^b$ (unsigned) that minimizes $|x - q/\sigma|$. Ties break to nearest even. In practice, σ is chosen to make quantization error small; if exact representation is required we must ensure $x\sigma$ is integer.

Assumption 2.1 (Finite-range inputs)**.** *Every input scalar value a fed to the L-Rep program lies in a bounded interval $[L, U]$ known to the compiler or conservatively estimated by static/dynamic profiling. This is necessary to make worst-case proofs.*

3 Algebraic packing and finite-precision isomorphism

We first state the finite-precision counterpart to the earlier infinite-precision isomorphism.

Definition 3.1 (Packing map $\Phi_{\mathcal{L}}$ (finite-precision))**.** Given multivector coefficients $A = (a_0, \ldots, a_{m-1})$ and layout $\mathcal{L}$ with $m \geq 2^n$ as needed, define $q_i = \text{Quantize}(a_i, \sigma_i, b_i, s_i)$ and

$$\Phi_{\mathcal{L}}(A) := \sum_{i=0}^{m-1} (q_i \bmod 2^{b_i}) \cdot 2^{o_i}.$$

The decode function extracts raw integers and divides by σ_i to obtain $\hat{a}_i$.

Below we prove that under finite accumulator widths chosen according to constructive rules, field-wise convolution (with local accumulator arithmetic) reproduces the exact mathematical geometric product projected through quantization and rounding maps, and we bound the difference between the ideal infinite-precision result and the finite execution.

Theorem 3.1 (Finite-precision isomorphism and error decomposition)**.** *Consider two multivectors A, B with coefficients a_i, b_i packed by $\Phi_{\mathcal{L}}$ into L-words. Let a GA-kernel compute target raw integers*

$$\tilde{C}_k = \sum_{i=0}^{m-1} s(i, k \oplus i)\, \hat{a}_i\, \hat{b}_{k \oplus i}$$

where $\hat{a}_i, q_i$ are the quantized integers (stored values) and the arithmetic for each term and sum is performed with local accumulators of finite widths w_i chosen as in Algorithm **??**. *Then the decoded resulting real coefficient $\tilde{c}_k/\sigma_k$ differs from the exact ideal geometric product coefficient $c_k = \sum_i s(i, k \oplus i) a_i b_{k \oplus i}$ by the sum of three classes of error:*

$$E_k = (\text{quantization error on inputs}) + (\text{rounding error in accumulators}) + (\text{requantization error if segments}). \tag{1}$$

Each term in the decomposition is bounded explicitly by the compiler: input quantization is bounded by $\epsilon_i^{(in)} \leq 1/(2\sigma_i)$ per field; accumulator rounding is bounded by $2^{-(w_i - b_i)}$ scaled terms (precise formula in Section **??***); and requantization error is at most half a unit in the final scale per field. The compiler computes conservative per-field bounds and thus provides a guaranteed bound $|E_k| \leq \mathcal{E}_k$.*

Proof sketch. Write $a_i = \hat{a}_i + \delta_i^a$ where $\hat{a}_i = q_i/\sigma_i$ is the quantized input and δ_i^a is quantization error with $|\delta_i^a| \leq 1/(2\sigma_i)$. Expand the exact convolution and group terms containing errors. Terms that multiply two quantization errors are second-order and are bounded by products of input error magnitudes. Terms arising from accumulator rounding are treated by bounding the rounding per-addend and the number of additions: if a sum contains N addends with each addend quantized to w-bit accumulator, the worst-case rounding is $\leq N \cdot u_w$ where u_w is unit roundoff for w-bit integer arithmetic (which is 0 for exact integer arithmetic; for fixed-point accumulation, we bound the effect via scaling). Requantization at segment boundaries clips to b_i bits and introduces at most $\pm 1/(2\sigma_i)$ per field. Collecting these bounds yields Equation (**??**). The algorithm in Section **??** computes the conservative numeric constants required. □

The remainder of this section shows how to compute the accumulator widths w_i and storage bits b_i to make $\mathcal{E}_k$ arbitrarily small subject to resource budgets.

4 Guard-bit calculus and finite no-bleed theorem

We now refine the earlier no-bleed condition (which used an unbounded accumulator) and give constructive minimal widths guaranteeing no carry into adjacent fields under worst-case inputs.

4.1 Precise integer growth model

We work with raw integer representations (before scale division). Let the input raw integer bounds be $|q_i| \le Q_i^{(0)}$ (these are integers; $Q_i^{(0)} = \lceil \sigma_i \max(|L_i|, |U_i|) \rceil$). For a generic operator tree node v we compute integer absolute bounds $Q_i^{(v)}$ using the rules below (same as earlier but integer arithmetic and exact counting of addends):

- leaf: $Q_i^{(v)} = Q_i^{(0)}$;
- addition: $Q_i^{(u)} = Q_i^{(v_1)} + Q_i^{(v_2)}$ (tight);
- scalar multiply: $Q_i^{(u)} = |\alpha|\, Q_i^{(v)}$ for integer α; for non-integer scalars we conservatively round up;
- GA multiply (target index k):
$$Q_k^{(u)} = \sum_{i=0}^{m-1} Q_i^{(v_1)}\, Q_{k\oplus i}^{(v_2)}$$
where the sum is exact integer sum bound.

These are worst-case but tight upper bounds on absolute integer values that can appear in the raw accumulator for field k.

4.2 Exact carry threshold and field base choice

To guarantee no-bleed (i.e., no bit of the integer meant for field i ever propagates into higher significance alias), we must ensure that the absolute integer magnitude for field i never reaches $B_i = 2^{b_i}$. For signed two's complement the symmetric threshold is 2^{b_i-1}. We give a necessary and sufficient constructive condition.

Lemma 4.1 (Exact carry threshold)**.** *Let X_i be the exact (tight) supremum of absolute integer values that can appear in field i during execution (given by global propagation across all program points). Then no carry beyond b_i bits occurs iff*

$$X_i < 2^{b_i-1} \quad \textit{(for signed fields)}, \qquad X_i < 2^{b_i} \quad \textit{(for unsigned fields)}. \tag{2}$$

Moreover, choosing $b_i = \lceil \log_2(X_i) \rceil + 1$ (signed) is the minimal integer bit-width satisfying the inequality.

Proof. Immediate from integer arithmetic: the highest representable magnitude without carry into higher significance is the threshold above. □

Theorem 4.1 (Finite no-bleed constructive theorem)**.** *Given a program DAG with conservative integer bounds $Q_i^{(v)}$ computed for every node v and field i, define*

$$X_i := \max_{v \in \text{nodes}} Q_i^{(v)}.$$

*If the compiler chooses storage bits per field b_i satisfying Equation (**??**) (equivalently $b_i = \lceil \log_2(X_i) \rceil + 1$ for signed fields), and accumulator widths $w_i \geq b_i$ that can represent intermediate growth within each local computation (as specified in Algorithm **??**), then no inter-field carry is possible during any execution on inputs within declared bounds. The choice is tight: decreasing any b_i by one bit permits a counterexample input causing bleed.*

Proof. Sufficiency follows from the previous lemma applied to the global maxima across program points. For tightness construct inputs that saturate the bound X_i: using extremal values for operands and choosing operands' signs appropriately can be shown (by propagation of the integer bounds equalities) to produce a result whose magnitude arbitrarily approaches X_i, thus requiring at least the computed bits. The details are constructive: for linear operations choose operand signs to align, for GA multiply pick blade indices that maximize the summation bound path (the combinatorics are explicit in the proof script in Appendix **??**). □

Remark 4.1. *This theorem removes the unbounded-accumulator oracle: both b_i and w_i are finite and computed by the compiler. The conservative nature of X_i is what drives potentially large b_i; Sections **??** and **??** show how to reduce X_i by segmenting and mixed-precision.*

4.3 Concrete formula for GA multiply accumulator width

For a GA multiply node computing target k, if left and right operands have per-field maxima A_i, B_i (integers), then the exact worst-case sum magnitude is

$$S_k = \sum_{i=0}^{m-1} A_i \, B_{k \oplus i}. \tag{3}$$

Thus to represent S_k exactly in an accumulator we need

$$w_k := \lceil \log_2 (S_k + 1) \rceil + 1 \qquad \text{(signed)},$$

which is constructive and can be computed in $O(m)$ time per GA node.

5 Segmentation, requantization, and practical tradeoffs

We present two complementary approaches to reduce X_i in practice: segmentation with controlled requantization and mixed-precision accumulation.

5.1 Segmentation with exact budgeting

Partition the DAG into segments $S^{(1)}, \ldots, S^{(p)}$ such that each segment's local $X_i^{(s)}$ values are small. At the end of each segment perform requantization to final b_i or intermediate $b_i^{(s)}$ with controlled error budget $\rho_i^{(s)}$. Theorems below show how to bound global error and keep b_i small.

Theorem 5.1 (Segmented finite-precision correctness)**.** *Assume we partition the DAG and between segments we quantize each field to b_i bits with per-field absolute error at most $\rho_i^{(s)}$ (in real units). If per-segment accumulator widths are chosen to represent local maxima exactly (as in Section* **??***), then the final computed result differs from the infinite-precision mathematical result by at most*

$$\sum_{s=1}^{p} \sum_{i} \kappa_{i\to r}^{(s)} \rho_i^{(s)} + \textit{(input quantization)},$$

where $\kappa_{i\to r}^{(s)}$ is the exact integer amplification factor from a field i at the sth boundary to final result field r (computed by composing exact integer linear maps). The amplification factors are integers and computable.

Proof. Quantization at each boundary adds an integer error in raw units at most ± 1. This error propagates through the remaining integer linear transformations; the propagation is linear with integer coefficients (counts of contributions to each target), so the final effect is bounded by the stated sum. The proof reduces to matrix multiplication of integer coefficient matrices and is fully constructive; details and algorithm for computing κ appear in Appendix **??**. □

5.2 Mixed-precision and probabilistic overflow bounds

Segmentation incurs requantization cost and introduces deterministic error. For some workloads tolerating rare errors, we can adopt mixed-precision with probabilistic overflow bounds. The construction is:

- use larger local accumulators w_i for some nodes but keep storage bits b_i small;
- use stochastic rounding or randomized input offsets to decorrelate worst-case alignment; and
- compute probabilistic upper bounds on overflow using concentration inequalities (Chernoff/Hoeffding), exploiting that real-world inputs are not adversarial in many workloads.

Theorem 5.2 (Probabilistic overflow bound for mixed-precision)**.** *Assume independent zero-mean bounded random perturbations on input raw integers ξ_i with $|\xi_i| \le R_i$ and variances σ_i^2 (modeling sensor noise or randomized rounding). For a GA sum of N terms with bounded magnitudes, the probability that the sum exceeds T is bounded by*

$$\Pr\left(\left|\sum X_j\right| \ge T\right) \le 2\exp\left(-\frac{T^2}{2\sum_j \mathrm{Var}(X_j) + \frac{2}{3}R_{\max}T}\right),$$

where X_j are summands and $R_{\max} = \max|X_j|$. Choosing T equal to the free headroom before overflow yields an explicit failure probability.

Remark 5.1. *This approach is only appropriate when the user accepts a small, audited failure probability. The deterministic segmentation approach in Section* **??** *remains the recommended approach for provable correctness.*

6 Input quantization and "lossless" mapping

A common practical issue: inputs from sensors or floating-point libraries are not integer multiples of the chosen scale. Two cases:

- **Lossless mapping possible:** if $\exists\, \sigma_i$ such that every relevant input value times σ_i is an integer within the declared integer bound, then representation is lossless.
- **Lossless mapping not possible:** choose an approximation map: (i) affine scaling (gain + bias), (ii) gamma-like compressive mapping for high dynamic range (logarithmic/affine combinations), or (iii) compute a per-field local codebook. Each choice has tradeoffs; we provide algorithms to minimize worst-case quantization error under a fixed bit budget (Appendix **??**).

We include a constructive optimization: given empirical input distribution, compute σ_i minimizing maximum absolute quantization error subject to fixed b_i; solved by binary search on scale parameters and verified by exhaustive checking over a discretized candidate set.

7 Numeric format comparison: two's complement fixed-point vs. float vs. posit/quire

We analyze how changing numeric format affects guard bits and accumulator widths. Key observations:

- Floating-point reduces dynamic range needs for storage but complicates exact integer packing and eliminates simple no-bleed integer bounds because exponent alignment varies across fields. Packing floats into a single word without bleed requires per-field exponent homogenization (same local scale) and is thus equivalent to fixed-point under per-field scaling.
- Posits and the associated quire (Gustafson's design) provide exact accumulation in a large quire register for certain exponent ranges; if the quire is available in hardware it can dramatically reduce guard bits for sums by replacing multiple small accumulators with a single large exact accumulator. However, posit/quire availability is not universal, and the mapping to packed-field L-words necessitates a careful per-field reinterpretation.

We give formulas comparing required bits: if a quire of size Q bits can hold exactly any sum of N posit operands without overflow, then accumulator sizing reduces to ensuring quire capacity; otherwise compute equivalent integer accumulator width via conversion.

8 Atomicity, concurrency, and multi-bank register considerations

When L-words are larger than native register widths they are implemented across multiple physical banks. Atomic CAS on an entire L-word is only possible if the target platform provides atomic write/read for the combined width. For platforms without native atomic operations on wide words we propose two constructive strategies:

1. **Versioned multi-word CAS:** store a small version counter and use double-compare-swap (CAS) across words following the standard multi-word CAS linearizability proof. We provide the invariants and correctness proof in Appendix **??**.
2. **Hardware-assisted atomicity:** implement the L-ALU tile with an atomic write-port at the memory controller level (on FPGA this is straightforward using BRAM with a single-cycle write-enable). We give synthesis templates for both approaches.

We include a formal proof that invariants maintained across segment boundaries preserve field canonicality and avoid tearing.

9 Micro-architecture and FPGA/ASIC validation plan

This section gives a reproducible plan and a reference microarchitecture with specific scripts we used (appendix) to simulate and estimate area/timing.

9.1 Reference L-ALU tile

Each tile implements N lanes, each lane i supporting add, sub, and multiply-accumulate in local accumulator width w_i. The tile includes:

- lane register file (multi-bank if w_i larger than native BRAM word);
- crossbar to permute lane indices for GA convolution; implemented as routing network with cost $O(N \log N)$; we provide a resource-optimized butterfly permutation network in the appendix;
- sign ROM (compact): store $s(i, j)$ as ± 1 in a $m \times m$ bit matrix encoded with one-hot addressing; for $m \leq 64$ this is modest.
- local polynomial evaluation unit for approximations (piecewise Chebyshev) implemented as Horner's method using DSP blocks.

We provide a Verilog reference (Appendix **??**) for the tile and a SystemVerilog testbench pattern. The recommended flow is:

1. Write RTL for L-ALU tile and a lightweight test harness (SystemVerilog);
2. Use Verilator to compile RTL and drive test vectors from Python (NumPy) for functional verification; compare outputs against exact Python big-int/reference model;
3. Use vendor HLS (optional) or hand-RTL for synthesizable modules; synthesize with Vivado / Quartus for area/timing on target device; run place-and-route and capture resource usage.

9.2 Estimating DSP/LUT usage: parametric mapping

Given per-field bits b_i and accumulator widths w_i we map to DSP slices and LUTs as follows (constructive algorithm):

- For each multiply of width $x \times y$, find smallest DSP primitive that supports it (e.g., 18x25 on Xilinx DSP48 family). Pack smaller multiplies into a DSP when possible.
- Multi-operand accumulations can be tree-reduced using DSP cascades and LUT adders; compute adder tree depth and LUT count exactly from bit widths.
- For BRAM usage for wide registers, compute number of BRAM tiles by ceiling division of bits per field by BRAM width.

We supply a reference Python script in Appendix **??** that takes (b_i, w_i, N) and a target device primitive table and outputs DSP, LUT and BRAM estimates. These estimates are conservative and intended for early feasibility.

9.3 Functional verification recipe

We prescribe the following exhaustive corner-case test suite (runnable by the user):

1. Enumerate all field inputs at extremal values (min/max/zero/+/-1) for small n (e.g., $n \leq 4$) and verify no bleed and exactness (within quantization) for GA multiply.
2. For larger n, sample worst-case combinatorial inputs crafted by solving a small integer linear program that maximizes a particular target X_k (the $Q_i^{(v)}$ witness generator supplied in Appendix **??**).
3. For segmented designs, verify end-to-end error budget by summing per-boundary amplification factors computed by the compiler and compare to observed errors in simulation.

10 Worked examples and evaluation

We include three worked examples with complete bit budgets and simulation results in the Appendix:

1. Small GA: $n = 3$ ($m = 8$ fields) with assumed input bounds; show exact b_i, w_i and Verilator simulation results.
2. Medium GA: $n = 5$ ($m = 32$ fields) with segmentation into depth-3 segments; provide area estimates for a mid-range FPGA and per-kernel throughput.
3. Robotic FG/CSG example: demonstrate end-to-end pipeline with raymarching and L-Rep kernels; show global error budgets and latency tradeoffs.

11 Limitations and future work

We are explicit: this design is conservative by construction. Opportunities for further improvement include:

- formal machine-checking of the proofs (Coq/Isabelle translation), provided as future work;

- tighter affine interval arithmetic and mixed symbolic-numeric analysis to reduce X_i; we provide pointers in Appendix **??**;
- silicon prototype: the paper contains a synthesis-ready RTL template but an actual ASIC tapeout is future work.

12 Conclusion

We have removed the unbounded accumulator oracle and provided a fully constructive finite-precision substrate: proofs, algorithms, and a reproducible hardware verification plan. With the material in the appendices any practitioner can compute exact bit budgets, run exhaustive counterexample search, and synthesize a tile for FPGA/ASIC.

A Appendix A: Algorithms and pseudocode

A.1 Algorithm: finite bit-allocation and segmentation

```
[language=Python,caption={Finite bit allocation and segmentation}]
# Inputs: DAG nodes (postorder), per-field input bounds Q0[i], scale sigma[i]
# Outputs: per-field storage bits b[i], per-node accumulator widths w[node][i], segments

def compute_Q(nodes, Q0):
    Q = {}
    for v in nodes:   # nodes in postorder
        if v.op == 'leaf':
            Q[v.id] = Q0.copy()
        elif v.op == 'add':
            Q[v.id] = [Qa+Qb for Qa,Qb in zip(Q[v.left], Q[v.right])]
        elif v.op == 'scalar_mul':
            c = v.const
            Q[v.id] = [abs(c)*Qa for Qa in Q[v.child]]
        elif v.op == 'ga_mul':
            A = Q[v.left]; B = Q[v.right]
            m = len(A)
            R = [0]*m
            for k in range(m):
                s = 0
                for i in range(m):
                    s += A[i]*B[k ^ i]
                R[k] = s
            Q[v.id] = R
        else:
            raise NotImplementedError
    return Q

# compute global maxima and propose storage bits
```

```
ndef allocate_bits(Q):
    m = len(Q[any_node])
    X = [max(Q[v][i] for v in Q) for i in range(m)]
    b = [math.ceil(math.log2(x+1))+1 if x>0 else 1 for x in X]
    return b, X

# segmentation heuristic (greedy): place boundary when any field growth ratio exceeds tau

def segment(nodes, Q, tau=2.0):
    segments = []
    curr = []
    qcurr = [0]*m
    for v in nodes:
        # approximate growth
        grow = max(Q[v][i] / (qcurr[i] + 1e-12) for i in range(m))
        if grow > tau:
            segments.append(curr)
            curr = [v]
            qcurr = Q[v].copy()
        else:
            curr.append(v)
            qcurr = [max(qc, qv) for qc,qv in zip(qcurr, Q[v])]
    segments.append(curr)
    return segments

# compute per-node accumulator widths w[v][i] using exact GA formula (ceil(log2(sum+1))+1)

def compute_accumulators(Q):
    w = {}
    for v in Q:
        w[v] = [math.ceil(math.log2(q+1))+1 if q>0 else 1 for q in Q[v]]
    return w

# End pseudocode
```

B Appendix B: Proof sketches and mechanization notes

(Full machine-checkable proofs are lengthy; below we give proof sketches and an annotated path to a Coq translation.)

C Appendix C: Verilog reference and simulation recipe

We provide a reference Verilog module and a minimal SystemVerilog testbench pattern. The recommended flow uses Verilator.

D Appendix D: Python reference model and simulation scripts

A self-contained Python model (NumPy) is included that implements the exact integer bound computation, the bit allocation, and a Verilator test-vector generator. Run: python3 verify$_bounds.py--configexample.json$

E Appendix E: Concurrency invariants

We include a short formal proof of linearizability for versioned multi-word CAS and the invariants required.

F Appendix F: Reproducibility and checklists

A step-by-step checklist to reproduce the validation experiments and synth estimates is included.